

答辩问题预测与参考答案

基于STM32与OneNet的智能室内环境监测与调控系统

答辩人:杨国豪 专业:电气工程及其自动化 嘉兴大学 2026届

一、研究背景与选题意义

Q1 你为什么选择这个课题？有什么研究意义？

研究表明人有超过70%的时间在室内度过，室内环境质量直接影响健康、工作效率和生活舒适度。传统监测存在三大痛点：

- ① 信息孤岛：依赖人工或单一设备，难以连续全方位感知；
- ② 响应滞后：参数异常时无法及时通知或采取措施；
- ③ 能耗浪费：设备无法按实时数据精细调节。

本课题结合STM32嵌入式技术、ESP8266 Wi-Fi模块和OneNet云平台，设计出低成本、支持实时监控与远程调控的完整闭环系统，具有较强的工程实用价值，适合智能家居、实验室、节能办公等多种场景。

Q2 目前国内外研究的不足在哪里？你的系统如何改进？

现有研究共性不足有三点：

- ① 只完成监测功能，缺少监测-分析-调控的完整闭环；
- ② 偏重底层硬件实现，用户交互界面设计不足；
- ③ 系统通用性差，难以跨场景快速移植和扩展。

本系统的改进：完整实现了采集→本地显示→云端同步→远程控制的闭环；以微信小程序为轻量化用户端，降低使用门槛；硬件模块化设计，可扩展性强，新增传感器不需改动已有模块代码。

二、系统设计与技术选型

Q3 为什么选STM32F103C8T6，而不是Arduino或ESP32？

STM32F103C8T6基于ARM Cortex-M3内核，主频72MHz，集成USART/I2C/SPI/ADC等丰富外设，SRAM 20KB，Flash 64KB，具备完善低功耗模式，更贴近工业实际开发。

对比Arduino：STM32主频(72MHz)远高于Arduino UNO(16MHz)，外设更丰富，驱动能力更强，工程实用性更强。

对比ESP32：将网络功能交给专用ESP8266处理，使主控专注于传感器采集和本地控制逻辑，实现功能解耦，后期也便于独立升级网络模块，架构更清晰。

Q4 为什么选MQTT协议而不用HTTP进行设备数据传输？

MQTT是为低带宽、不稳定网络和资源受限设备设计的轻量级消息协议，相比HTTP有以下优势：

- ① 报文开销极小(最小固定头仅2字节)，远小于HTTP头部，节省流量；
- ② 采用发布/订阅模式，支持一对多消息分发，解耦了设备端和应用端；
- ③ 支持三级QoS(0/1/2)，可根据需要保障消息可靠送达；
- ④ 使用长连接，服务器可主动推送指令，实时性远优于HTTP轮询。

本系统设备端用MQTT，微信小程序与平台之间用HTTP RESTful API，两者各得其所。

Q5 为什么选OneNet云平台？和阿里云IoT相比有什么优劣？

选择OneNet的主要理由：① 中国移动运营，国内网络稳定、延迟低；② 对学生开发者免费且配额充足；③ 原生支持MQTT/HTTP/TCP多种协议；④ 内置可视化仪表盘，快速验证功能无需额外开发；⑤ 提供开放API，方便小程序调用。

与阿里云IoT相比：阿里云功能更完善(规则引擎、设备影子、OTA等)，适合商业化产品；但对学生项目而言配置复杂、费用门槛较高。

综合开发周期、成本和功能需求，OneNet更适合本课题的学术验证目标。

Q6 DHT11精度够用吗？为什么不选精度更高的DHT22或SHT30？

DHT11标称精度温度 $\pm 2^{\circ}\text{C}$ 、湿度 $\pm 5\%\text{RH}$ ，经实测最大误差 0.5°C 、 $2\%\text{RH}$ ，优于标称值。对室内舒适度监测(人体感知温度变化通常在 1°C 以上)，该精度已够用。

选择DHT11的理由：① 成本极低(约1-2元)，符合低成本目标；② 单总线协议只需一根数据线，驱动简单；③ 内部已校准，直接输出数字量，无需外部ADC。

DHT22精度略高但成本更高；SHT30精度约 $\pm 0.3^{\circ}\text{C}$ 但价格约10倍以上。后续改进可升级为SHT30以提高精度。

Q7 GL5516光敏电阻与BH1750数字光照传感器各有什么优缺点？

GL5516(模拟型)：优点是成本极低(几毛钱)，电路仅需分压电路接ADC即可；缺点是输出电压与光照非线性，只能给出相对百分比(无lx绝对值)，受温度影响，个体一致性差。

BH1750(数字型)：优点是直接输出标准lx值，精度高(分辨率1 lx)，I2C接口简洁，线性好，温漂小；缺点是成本约为GL5516的5-10倍。

本系统选GL5516出于成本控制考虑，论文结论也指出了这一局限，后续改进方向就是替换为BH1750。

三、硬件电路设计

Q8 STM32最小系统由哪些电路组成？各部分作用是什么？

① 电源模块：VDD接3.3V，每个电源引脚并联 $0.1\mu\text{F}$ (滤高频)和 $10\mu\text{F}$ (滤低频)电容，保证供电稳定；

② 时钟模块：8MHz无源晶振配合两个 22pF 负载电容产生主时钟，PLL倍频至72MHz；另加32.768kHz晶振给RTC提供独立低功耗时基；

- ③ 复位模块:按键+10kΩ上拉电阻+0.1μF电容组成低电平有效复位电路,上电时电容缓慢充电保证可靠复位;
- ④ 调试接口:保留SWD两线接口(SWDIO+SWCLK),通过ST-Link完成程序下载和在线调试,比JTAG节省引脚。

Q9 蜂鸣器驱动电路为什么要用三极管?直接用GPIO驱动不行吗?

STM32 GPIO引脚最大输出电流约25mA,而本系统选用的5V有源蜂鸣器工作电流约30mA,且工作电压5V,超过STM32 GPIO驱动能力。

因此使用NPN三极管(S8050)作为开关管:GPIO输出3.3V高电平,通过1kΩ基极限流电阻驱动三极管导通,集电极接蜂鸣器负极,5V电源经蜂鸣器流向三极管再到地,蜂鸣器鸣响。PB1低电平则三极管截止,蜂鸣器断电。

这一方案同时实现了电平转换(3.3V控制5V设备)和电流放大,保护了STM32芯片。

Q10 MQ-2为什么需要预热20秒?预热期间系统如何处理?

MQ-2采用SnO₂半导体气敏材料,内部电热丝需将材料加热至约200-300℃最佳工作温度,气敏特性才能稳定。预热不充分时输出不稳定,误差大。

系统处理方式:上电初始化阶段延时约20秒等待预热;预热期间OLED显示提示信息,不向云端上报烟雾浓度数据;预热完成后方正常采集上报,保证了数据的有效性。

四、软件设计与通信实现

Q11 DHT11单总线通信时序如何实现?如何保证时序准确?

通信分三阶段:

- ① 起始信号:MCU拉低数据线至少18ms,再拉高等待20-40μs触发传感器;
- ② 传感器响应:DHT11拉低80μs响应,再拉高80μs准备发送;
- ③ 数据传输:发送40位数据(湿度整数+小数+温度整数+小数+校验和),每位以50μs低电平开始,后跟高电平——26-28μs表示逻辑0,70μs表示逻辑1。

准确性保障:使用SysTick定时器实现微秒级精准延时(DelayUs);每次通信后验证校验和(前四字节之和等于第五字节),校验失败则重新采集。

Q12 主循环如何调度多任务?有没有考虑使用FreeRTOS?

本系统采用非阻塞式时间片轮转调度:主循环依次执行传感器采集(约3ms)→OLED刷新→数据打包→MQTT上报→指令接收解析,单次循环总耗时约5ms,实时性良好。

关于FreeRTOS:本系统任务数量少(约5个),任务间无严格时序依赖,裸机轮询调度已能满足需求。引入FreeRTOS会增加内存开销和开发复杂度,对本系统属于过度设计。

若后续扩展任务数量或需要优先级管理,可引入FreeRTOS将各任务封装为独立Task,利用信号量和队列通信。

Q13 ESP8266 AT指令通信流程是怎样的？超时了如何处理？

初始化流程:AT测试→设置STA模式→启用DHCP→连接Wi-Fi热点→与OneNet建立TCP连接→发送MQTT CONNECT包完成云端握手。

通信实现:STM32通过USART2(PA2/PA3, 115200bps)发送AT指令字符串, 中断接收响应存入缓冲区, strstr()检测关键字(OK/CONNECT等)判断执行结果。

超时处理:每条AT指令设置200次×10ms=2秒超时, 超时后自动重发;连接类指令持续重试直到成功;数据上报类超时则丢弃本次数据, 等待下一周期, 避免程序卡死。

Q14 JSON数据如何打包上报和解析下行指令？为什么用cJSON库？

上报格式(符合OneNet物模型):将四路传感器数据封装为JSON, 包含id、version、params字段, params中含Temp/Hum/light/Amoke及其value, 通过MQTT发布到指定主题。

指令解析:收到MQTT Publish后, cJSON_Parse()解析payload, cJSON_GetObjectItem()逐级获取params下的LED/Alarm字段, 根据布尔值控制GPIO高低电平。

选择cJSON的原因:① 纯C实现, 无外部依赖, 可直接移植到STM32裸机;② 内存占用小, API简单;③ 开源稳定, 使用广泛;④ 支持动态内存, cJSON_Delete()释放, 不造成内存泄漏。

五、测试结果与性能分析

Q15 温度误差±0.5℃, 你是如何测试得出的？方法是否科学？

测试方法:选用精度±0.1℃的高精度数字温度计作为参照, 在三种典型环境(空调房约20℃、室内自然环境约25℃、封闭空间约30℃)下同步采集, 每种环境采集10组数据取平均值, 计算绝对误差。

结果:温度最大绝对误差0.5℃、湿度最大2%RH, 优于DHT11手册标称精度, 说明时序驱动和校验机制有效。

局限性:样本量有限(30组), 温湿度范围未覆盖极端工况, 参照仪器非计量认证级别。但对学术验证目的而言, 方法具有合理性和可重复性。

Q16 数据上传成功率99%, 失败的1%是什么原因？如何改进？

失败原因:主要发生在网络信号波动时段, 包括:Wi-Fi热点短暂断开导致TCP连接中断;路由器DHCP重新分配IP后原连接失效;MQTT Broker短暂不可达。

当前机制:发送失败后在下一采集周期重试, 已实现基本重传, 未出现数据永久丢失。

改进方向:① 增加MQTT心跳包(Keepalive)主动检测连接状态;② 实现本地数据缓冲, 网络恢复后批量补传;③ 实现断线自动重连逻辑(已在展望中提出), 消除对手动复位的依赖。

Q17 远程控制响应时间2.3秒, 延迟来自哪里？能否降低？

延迟来源分解(约2.3秒):

- ① 小程序HTTP POST到OneNet服务器:约0.3-0.5s(手机网络+服务器处理);
- ② OneNet平台处理并MQTT下推到设备:约0.5-0.8s(平台内部转发);

③ ESP8266接收并通知STM32:约0.1-0.2s;

④ STM32解析JSON并执行GPIO:小于10ms(可忽略)。

降低延迟的措施:将小程序轮询改为WebSocket长连接;使用QoS 0降低握手开销;升级至延迟更低的云平台。2.3秒对手动控制场景已是可接受水平。

Q18 系统总功耗265mA, 如何评价? 有哪些优化方案?

功耗构成:MQ-2约150mA(内部电热丝, 占比最大)、ESP8266约70mA、STM32约25mA、OLED约20mA, 合计约265mA(5V供电约1.3W)。

评价:有线供电场景合理;电池供电场景则续航较短, 需要优化。

优化方案:① ESP8266启用Deep Sleep, 仅上报时唤醒, 休眠电流小于1mA;② 降低上报频率(秒级降至分钟级), 减少ESP8266工作时长;③ MQ-2间歇供电;④ STM32非采集期间进入低功耗模式。综合优化后平均电流可降至50mA以内。

六、系统不足与展望

Q19 Wi-Fi断开后需手动复位, 为什么没实现自动重连? 如何改进?

原因:当前软件在初始化时完成一次性连接, 断网后主循环中发送失败被记录但未触发重连流程, 初始化函数中的连接逻辑未被设计为可复用的状态机。

改进方案:① 添加MQTT心跳包机制, 超时未收到响应则判定断线;② 监测ESP8266串口返回的断线通知;③ 断线后重新执行Wi-Fi连接→TCP建立→MQTT CONNECT→重新订阅主题的完整恢复流程。

这一功能实现并不复杂, 是后续版本的优先改进项, 预计只需增加约50行代码。

Q20 系统后续如何扩展, 比如增加CO2或甲醛传感器?

CO2传感器(如MH-Z19B):采用UART串口通信, 输出400-5000ppm数字值, 接STM32的USART3即可, 无需ADC, 驱动简单。

甲醛传感器(如ZE08-CH2O):支持UART和模拟两种输出, 推荐UART模式获取更精确数值。

扩展步骤:① 硬件增加传感器模块, 占用额外UART或ADC引脚;② 软件新增对应驱动, 主循环加入采集逻辑;③ OneNet物模型增加新数据流定义;④ 微信小程序界面增加对应显示卡片和阈值报警。

模块化设计保证了扩展不影响已有代码, 只需新增, 不需修改。

七、综合与开放性问题

Q21 开发过程中遇到的最大技术难点是什么? 怎么解决的?

最大难点:ESP8266与OneNet云平台的MQTT连接调试。

具体问题:① AT指令序列繁多(7步初始化), 任何一步失败都导致后续无法进行, 错误现象相似难以定位;② OneNet MQTT鉴权格式文档描述不清晰;③ 接收OneNet下发的MQTT PUBLISH消息时出现粘包问题。

解决方法:① 通过USART1串口输出详细日志, 逐步打印每条AT指令的发送和响应;② 仔细阅读OneNet官方文档并参考社区案例, 确认token生成方式;③ 在串口接收中断中实现基于报文长度字段的帧解析, 解决粘包。

这一过程让我深刻理解了分层调试思路的重要性, 也积累了大量物联网终端开发的实践经验。

Q22 如果将这个系统产品化, 还需要解决哪些问题?

产品化还需解决:

- ① 安全性: 当前MQTT未启用TLS加密, 需增加传输加密和设备唯一身份认证;
- ② 可靠性: 增加看门狗定时器防程序跑飞; 完善断网重连; 增加本地数据缓存;
- ③ 一致性: GL5516等传感器个体差异大, 产线需逐台标定;
- ④ 认证合规: 需通过EMC/EMI电磁兼容认证, PCB布局需专门优化;
- ⑤ 用户功能: 微信小程序需完善注册/登录、设备绑定、历史查询、阈值推送等完整功能;
- ⑥ 成本控制: 规模化生产时需对元器件选型再优化, 进一步降低BOM成本。

Q23 本次毕业设计你最大的收获是什么?

三个层面的收获:

- ① 技术层面: 将四年所学的嵌入式原理、单片机应用、通信原理、传感器技术融会贯通, 第一次独立完成了从需求分析、硬件设计、嵌入式编程到云平台配置、小程序开发的完整系统, 深刻理解了IoT端-管-云架构的实际运作方式。
- ② 工程思维: 学会了系统化调试——遇到问题先缩小范围、分层排查; 理解了文档和测试在工程中的重要性。
- ③ 综合素养: 通过论文写作训练了科学表达和逻辑组织能力; 也学会了客观评估自身工作的局限性, 论文的不足部分写得比优点更让我印象深刻。

这次经历让我对毕业后从事嵌入式/物联网相关工作有了更清晰的方向和信心。

答辩小贴士

时间控制	汇报控制在8-10分钟, 重点讲第2、3部分(系统设计+测试结果), 背景和结论可适当压缩。
回答技巧	不知道的问题不要乱猜, 坦诚比乱答更好, 可以说还有待进一步研究, 然后说出自己的初步判断。
高频考点	重点准备: MQTT vs HTTP原理区别、STM32选型理由、DHT11通信时序、功耗分析、系统不足与改进方向——几乎必问。
主动引导	汇报时在第3部分多停留, 展示测试数据截图, 引导老师问你已准备好答案的问题。

祝答辩顺利!

本文档共 23 题, 覆盖背景、设计、硬件、软件、测试、展望、综合七大模块